

Задръствания и управление
на потоците в мрежата на 2,
3 и 4 ниво

Задръстване и ресурси

Контролът за задръстване (**Congestion control**) в мрежите и заделянето на ресурси (**resource allocation**) са двете страни на една и съща монета.

Прецизното заделяне на ресурси е трудна задача, защото въпросните ресурси (**буфери, пропускателна способност на линиите**) са разпръснати по цялата мрежа.

Задръстване и ресурси (2)

Можете да изпращате в мрежата, колкото си искате пакети, да ги възстановявате, но това ще наруши нормалната работа.

Затова се налага да прилагате механизми за контрол на задръстванията, т.е да има някаква предсказуемост на процесите.

Какво включва Congestion Control

Контролът за задръстване и заделянето на ресурси включват и крайните устройства (хостове), и мрежовите (рутери и др.).

При **мрежовите устройства** могат да се прилагат различни политики за подреждане в **опашките**, при запълване на буферите и предаване по линиите, т.е кои пакети да се предават, кои да се изхвърлят.

Какво включва Congestion Control (2)

При **хостовете** контролът за задръстване се отнася до това до **колко бързо на източниците се разрешава да изпращат пакети**.

За да не се случи задръстване, а ако се случи – лесно да бъде преодоляно.

Терминология

resource allocation – Процесът, чрез който мрежовите елементи се опитват да посрещнат изискванията на различните приложения по отношение на **скорост на линии, буферно пространство, мощност на процесора** в рутери и суичове.

congestion control – усилията, полагани от мрежовите възли да предпазят или да реагират на в случаи на **свърхнатоварване**.

Терминология (2)

Това може да се постигне най-просто чрез забрана на някои хостове да предават.

За да има справедливост, повечето механизми включват в себе си и **resource allocation**.

Flow control vs. Congestion control

Управлението на потока предпазва един “бърз” изпращач да препълни “бавен” получател.

Контролът на задръстванията предпазва множество от изпращачи (цялата мрежа) да изпращат много данни в мрежата при недостиг на ресурси.

Двете концепции споделят някои общи механизми.

Класическо “тясно място”

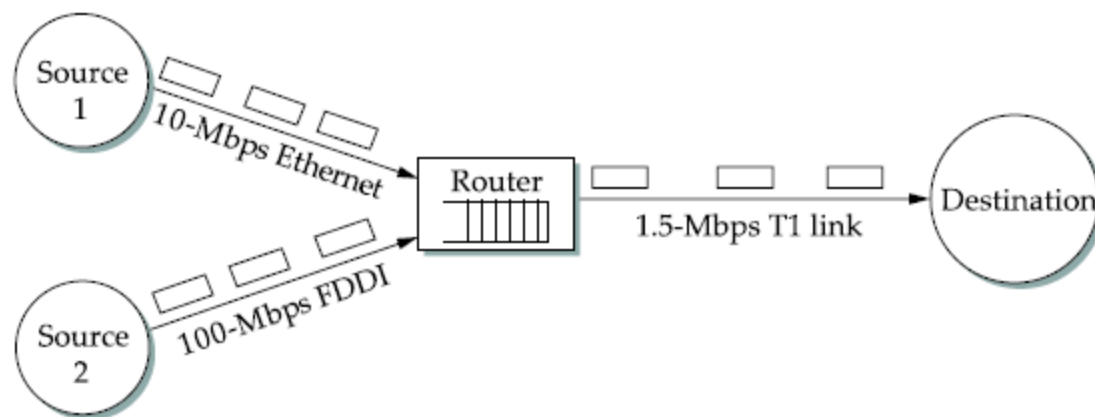


Figure 6.1 A potential bottleneck router.

Две високоскоростни линии “захранват” нискоскоростна. Рутерът може да бъде задръстен и става “тясното място” (**bottleneck router**).

Resource Allocation схеми:

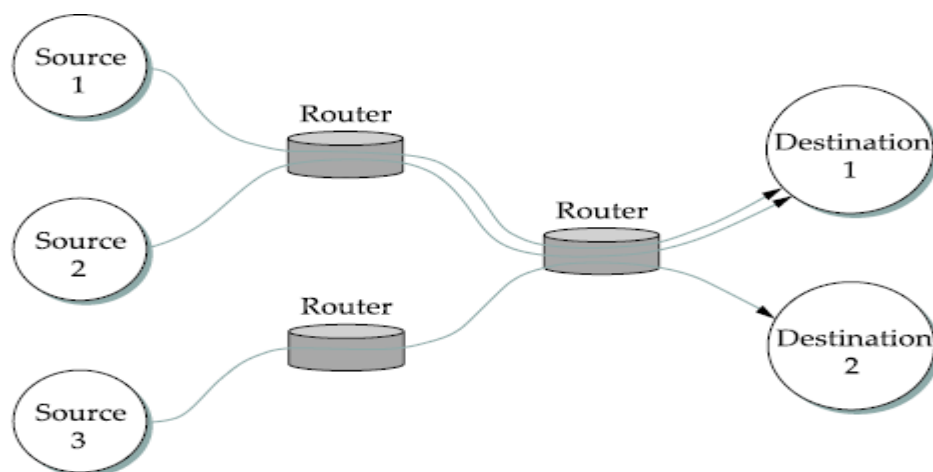


Figure 6.2 Multiple flows passing through a set of routers.

Потокът може да е **host-to-host** (един и същ source/destination IP) или **process-to-process** (един и същ source/destination port). Това може да се нарече канал.

Потокът е видим за рутера, а **каналът е end-to-end**.

Router-Centric vs. Host-centric.

Router-Centric – всеки **рутер** отговаря дали да препрати пакета или да го изхвърли и да информира хостовете колко пакета им е разрешено да изпращат.

Host-centric – крайните устройства (хостове) наблюдават колко пакета са минали успешно през мрежата и сами си настройват “поведението”.

Reservation-Based vs. Feedback-Based

Reservation-based – хостът пита мрежата за ресурси. Подобно е на телефона – дава “свободно” или “заето”.

Feedback-based – хостът изпраща данни без “предварителна резервация” и след това си нагласява скоростта в зависимост от обратната връзка:

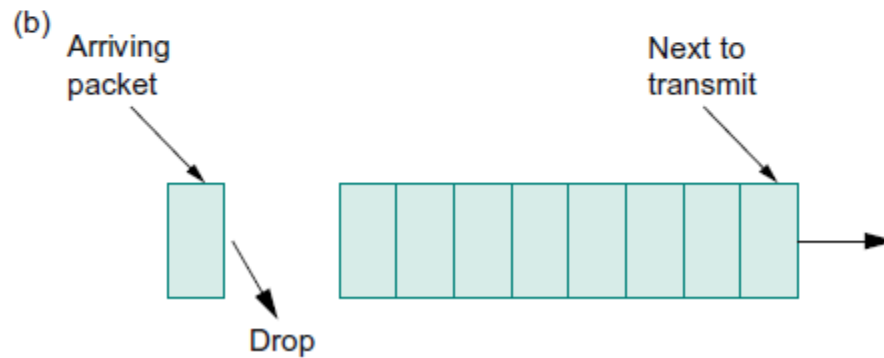
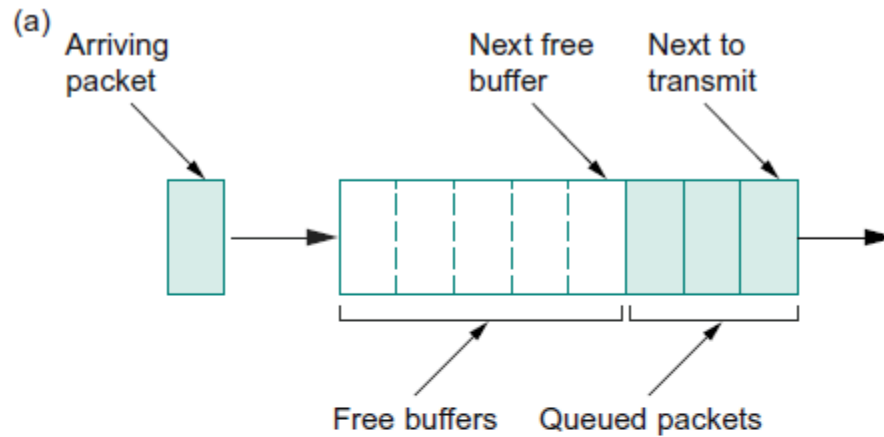
- експлицитно - “**please slow down**” или
- имплицитно - в зависимост от **положението на мрежата** (напр. Загуба на пакети).

Window-Based vs. Rate-Based

Прозоречният механизъм ни е известен от TCP и каналния слой – поддържа управлението на потока.

Да се контролира поведението на изпращача чрез скоростта: колко bits per second може да поеме получател или мрежата. Има отношение към качеството на услугата (QoS).

Resource Allocation. Опашките: FIFO



FIFO

FIFO опашки (queuing) с политика first-come-first-served (FCFS).

Механизмът tail drop (изхвърляне на пакет - случай (b) на предишния слайд) широко се използва от рутерите в Интернет.

Вариант на FIFO са опашките с приоритети. Всеки IP пакет да има приоритет, например указан в полето Type of Service (TOS). Получават се множество FIFO опашки с различни приоритети.

Fair Queuing

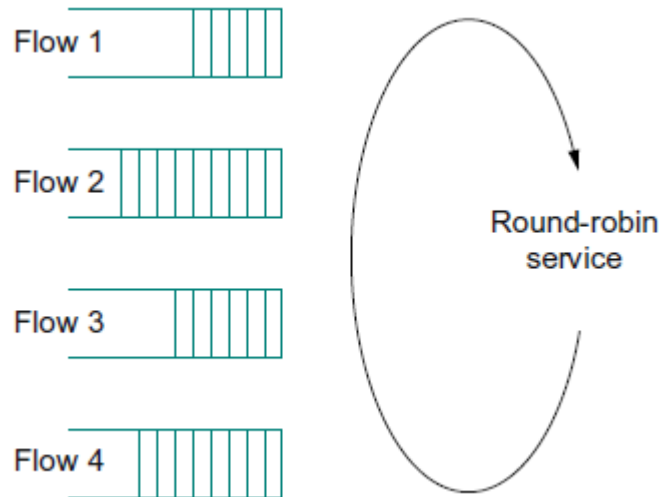
В Internet не всички приложения са TCP-базирани, т.е не ползват механизмите за управление на потока.

И е възможно да задръстят рутерите.

Тук на помощ идват “справедливите” опашки (Fair queuing – FQ): поддържа отделна опашка за всеки поток през рутера.

Рутерът ги обслужва в цикличен ред (round-robin).

Round-robin



Когато даден поток изпраща много бързо, опашката се запълва и при определен праг пакетите започват да се изхвърлят.

Така съответният източник не може да запълни мрежата за сметка на други потоци.

Weighted Fair Queuing

Вариант на FQ е **WFQ** – на всеки поток (опашка) се присвоява **тежест** (weight) – колко бита да се предават, когато рутерът обслужва опашката.

Простата FQ дава тежест 1 на всеки поток, логически да предава само по 1 bit, когато дойде реда.

WFQ, ако тежестта е 2, ще предава 2 бита и т.н.

Weighted Fair Queuing

WFQ може да се приложи на “**класове**” от трафик

Например, полето **TOS** в IP хедъра идентифицира класовете и служи за алокиране на опашка и тежест за всеки клас.

Това е част от архитектурата Differentiated Services (**DiffServ**).

TCP Congestion Control

TCP Congestion Control ([RFC 5681](#)) е въведен в края на 1980-те от [Van Jacobson](#).

Идеята на TCP CC е всеки източник да определя колко е свободния капацитет на мрежата, колко пакети може да изпрати.

Получаването на ACK е сигнал, че един пакет е напуснал мрежата и може да се въведе нов без опасност от задръстване.

Определянето на свободния капацитет не е лека задача, има много потоци в мрежата.

TCP Congestion Window

Congestion Window (**cwnd**) ограничава количеството данни, които TCP може да изпрати. TCP **не трябва** да изпраща данни със:

$\text{SeqNo} > (\text{AckNoMax} + \min(\text{cwnd}, \text{rwnd}))$.

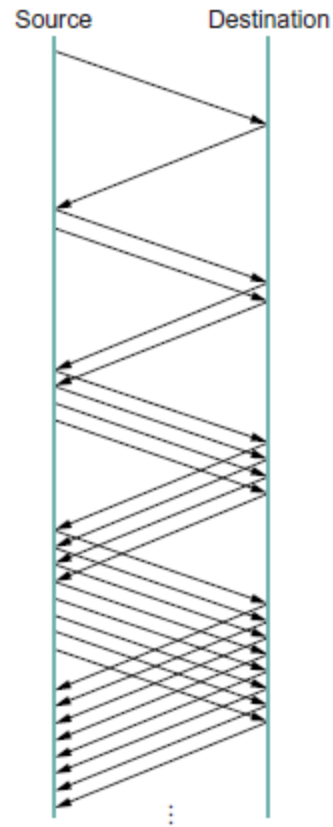
rwnd – Най-скоро рекламирания **receiver window**

Алгоритми за контрол на задръстването

RFC 5681 дефинира следните **алгоритми**: бавен старт, избягване на задръстването, бързо повторно предаване и бързо възстановяване.

Алгоритъм на **бавния старт (slow start)**. Започвайки да предава при неизвестни условия, ТСР **бавно пробва мрежата**, за да определи какъв е капацитета и да избегне задръстване с големи обеми от данни.

Бавен старт



Бавен старт. Начален прозорец.

IW, началната стойност на **cwnd**, приема една от следните стойности:

Ако SMSS (Sender MSS) > 2190 bytes:

$$IW = 2 * SMSS \text{ bytes}$$

Ако (SMSS > 1095 bytes) и (SMSS <= 2190 bytes):

$$IW = 3 * SMSS \text{ bytes}$$

Ако SMSS <= 1095 bytes:

$$IW = 4 * SMSS \text{ bytes}$$

Прагът на бавния старт

Началната стойност на `ssthresh` (slow start threshold) се слага произволно висока (напр., възможно най-големият рекламиран прозорец), но при задръстване се редуцира.

При висока стойност на `ssthresh` условията в мрежата, а не някой хост ще определят скоростта.

След бавния старт

В RFC 5681 се препоръчва `cwnd` да се увеличава с `min (N, SMSS)`,

`N` е броят на непотвърдените по-рано байтове, които са потвърдени от настоящия ACK.

Бавният старт приключва, когато `cwnd` надвиши прагово ниво `ssthresh` или при настъпване на задръстване.

Избягване на задръстването

При този алгоритъм `cwnd` се инкрементира с по един цял сегмент на всеки период round-trip time (`RTT`).

И продължава, докато се забележи задръстване.

Fast Retransmit/Fast Recovery

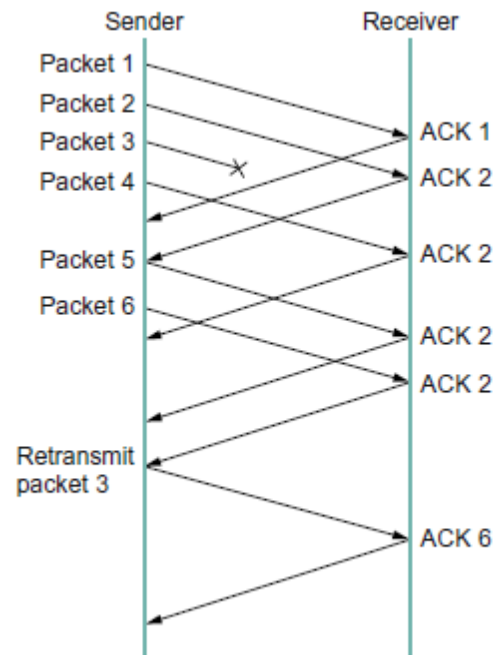
На следния слайд е илюстрирано как дублирани ACKs водят до бързо повторно предаване.

Дестинацията получава сегменти 1 и 2, но сегмент 3 се губи и дестинацията ще изпрати дублиран ACK за сегмент 2, когато пристигне 4, после след 5 и т.н.

Изпращачът вижда третия дублиран ACK за пакет 2 – в отговор на сегмент 6, и повторно предава сегмент 3.

Когато в дестинацията пристигне повторно предадено копие на сегмент 3, приемникът изпраща към източника сумарен ACK за всичко получено до момента, включително сегмент 6.

Fast Retransmit/Fast Recovery



RFC 5681 updated by RFC 9438

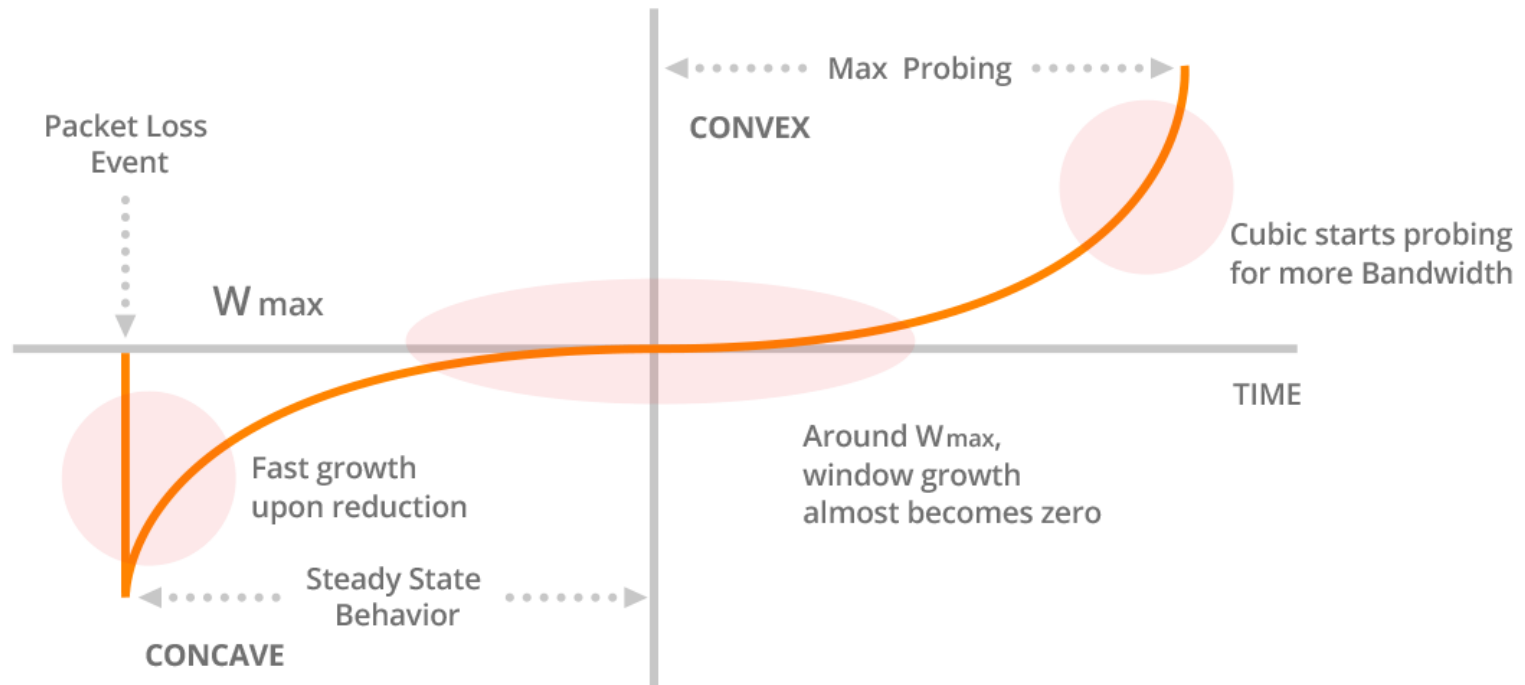
(CUBIC for Fast and Long-Distance Networks)

CUBIC е усъвършенстван алгоритъм за контрол на задръстванията в TCP протокола, предназначен да оптимизира работата на мрежи с висока пропускателна способност и голямо закъснение ("**Long Fat Networks**", напр. тези по дъното на океана).

Днес това е **стандартният избор по подразбиране** в основни операционни системи като Linux, Windows и macOS.

```
[RockyLinux ~]$ sysctl net.ipv4.tcp_congestion_control  
net.ipv4.tcp_congestion_control = cubic
```

Алгоритъм CUBIC



CUBIC

(кубична математическа функция)

За разлика от TCP Reno (RFC 5681), който увеличава прозореца линейно,

CUBIC използва кубична математическа функция за регулиране на **cwnd** въз основа на времето, изминало от последното събитие на претоварване.

Прозорецът на задръстване се изчислява по следната формула:

$$W(t) = C(t - K)^3 + W_{max}$$

CUBIC

(кубична математическа функция)

$W(t)$: Размерът на прозореца в момент t .

C : Константа на мащабиране (обикновено 0.4), която определя агресивността на растежа, **$C = \text{segment}/s^3$**

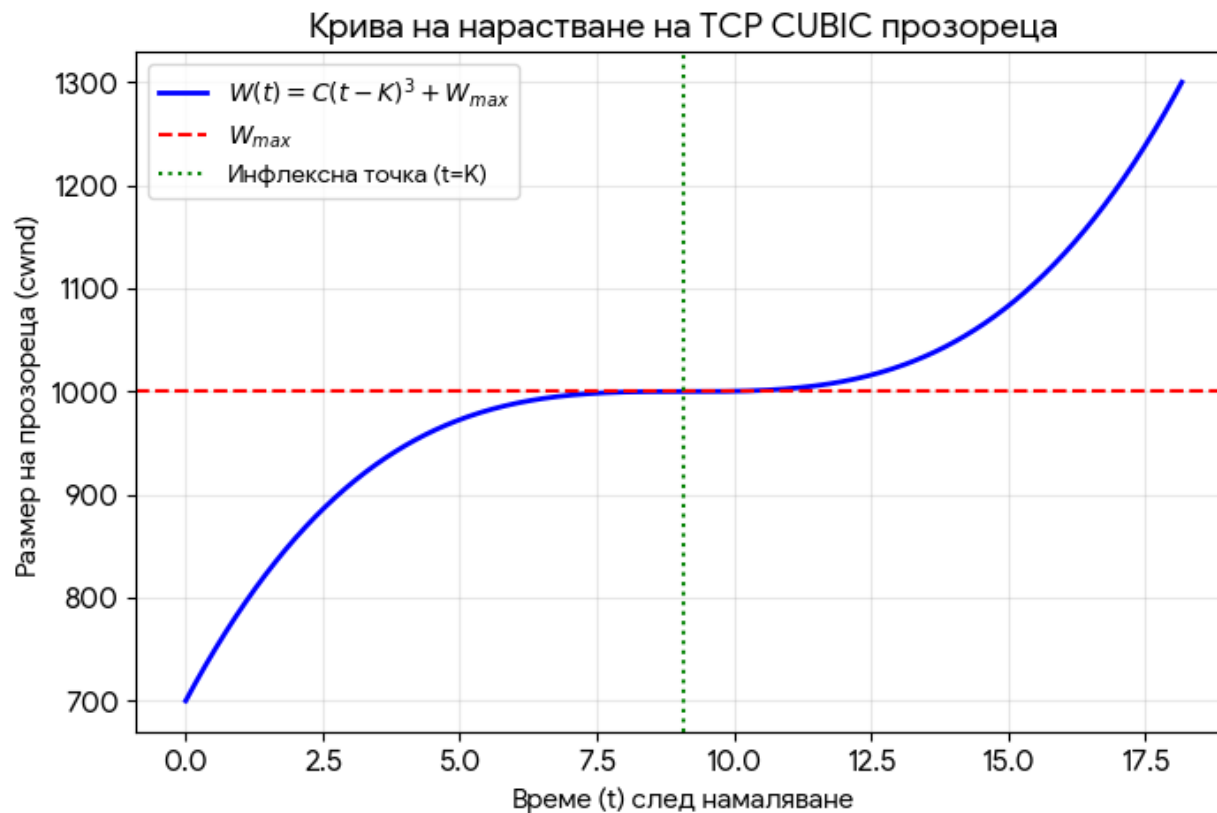
t : Времето, изминало от последното намаляване на прозореца.

W_{max} : Размерът на прозореца непосредствено преди последното събитие на задръстване.

K : Времевият период, необходим на функцията да достигне отново W_{max} .

CUBIC

(трите фази)



CUBIC (трите фази)

Процесът преминава през три основни фази след регистриране на загуба на пакет, като функцията създава "**S-образна**" крива на растеж с три ключови фази:

Вдлъбнатата фаза (**Concave region**): Веднага след загуба на пакет, прозорецът расте бързо в началото, но скоростта на растеж намалява, докато се приближава към предишния максимум ($W_{\max}$).

Плато (**Plateau**): Когато прозорецът е близо до $W_{\max}$, растежът е почти нулев. Това позволява на мрежата да се стабилизира при капацитет, за който вече е известно, че е достижим.

CUBIC (трите фази и предимства)

Изпъкнала фаза (**Convex region**): Ако след преминаване на $W_{\max}$, няма нова загуба, алгоритъмът започва агресивно да търси нова свободна честотна лента, като увеличава скоростта на растеж.

Независимост от RTT: Тъй като растежът зависи от реалното време, а не от честотата на пристигане на потвържденията (ACKs), потоците с различно забавяне споделят честотната лента по-справедливо.

Мащабируемост: Ефективно използва съвременни 10Gbps+ връзки, където стандартният Reno алгоритъм би отнел часове, за да възстанови пълната си скорост след една загуба.

TCP-Friendly: CUBIC включва режим на съвместимост, който гарантира, че не работи по-бавно от традиционния TCP Reno в стандартни мрежови условия.